

```

#include <stdio.h>
#include <string.h> // Required for strcspn int
main()
{ int i, x, b; char str[50]; printf("Program
By Shubham Patil A_29\n"); printf("\nPlease
enter a string: ");
    // CORRECTION 1: Replaced unsafe gets() with fgets()
    fgets(str, sizeof(str), stdin);
    // CORRECTION 2: Remove the newline character that fgets() stores str[strcspn(str,
"\n")] = '\0';

    printf("Enter the Key: ");
    scanf("%d", &b);

    printf("\nPlease choose following options:\n"); printf("1
= Encrypt the string.\n"); printf("2 =
Decrypt the string.\n"); scanf("%d", &x);

    // using switch case statements
    switch(x)
    { case 1: for(i = 0; i < 50 && str[i]
!= '\0'; i++)
        {
            str[i] = str[i] + b;
        }
        printf("\nThe Encrypted string: %s\n", str);
    break; case 2: for(i = 0; i < 50 && str[i]
!= '\0'; i++)
        {
            str[i] = str[i] - b;
        }
        printf("\nThe Decrypted string: %s\n", str); break;
    default:
        printf("\nError\n");
    }
    return 0;
}

```

```

#include <stdio.h>
#include <string.h> #include

```

```

<ctype.h> int isPresent(char letter, char key[],
int keyLen) {    int i;
    for (i = 0; i < keyLen; i++) {        if (key[i] == letter ||
(letter == 'J' && key[i] == 'I')) {
return 1;
    }
}
return 0;
}

int row(char letter, char playFairMatrix[5][5]) {
int i, j;
    for (i = 0; i < 5; i++) {
for (j = 0; j < 5; j++) {
    if (playFairMatrix[i][j] == letter || (letter == 'J' && playFairMatrix[i][j] == 'I')) {
return i;
    }
}
}
return -1;
}

int column(char letter, char playFairMatrix[5][5]) {
int i, j;
    for (i = 0; i < 5; i++) {
for (j = 0; j < 5; j++) {
    if (playFairMatrix[i][j] == letter || (letter == 'J' && playFairMatrix[i][j] == 'I')) {
return j;
    }
}
}
return -1;
}

void createPlayFairMatrix(char playFairMatrix[5][5]) {
char key[100];
    char keyWithoutDuplicates[26];
int i, j;    char letter = 'A';    int
keyIndex = 0;
    printf("\nProgram Performance of encryption and decryption using Play Fair
Cipher.\n");    printf("Execution By Shubham Patil A-29 \n");    printf("Please Enter the key:
");
fgets( key, sizeof( key), stdin);    key[strlen(key, "    \n")] = ' \ 0';    for (i = 0; key[i]
!= ' \ 0'; i++) { char    upperChar = toupper(key[i]);        if (isalpha(upperChar) &&
!isPresent( upperChar, keyWithoutDuplicates, keyIndex)) {
keyWithoutDuplicates[keyIndex++] =
upperChar;    }    }    keyWithoutDuplicates[keyIndex] = ' \ 0';    keyIndex = 0;    for (i =
0; i < 5; i++) {        for (j = 0; j < 5; j++) {            if (keyIndex < strlen(
keyWithoutDuplicates)) {                playFairMatrix[i][j] = keyWithoutDuplicates[
keyIndex++];

```

```

        } else {
            while (isPresent(letter, keyWithoutDuplicates, strlen(keyWithoutDuplicates))) {
letter++;
            }
            if
(letter == 'J') {
letter++;
            }
            playFairMatrix[i][j] = letter++;
        }
    }
}
}

```

```

void displayPlayFairMatrix(char playFairMatrix[5][5]) {
int i, j;
    printf("\nThe Play Fair Matrix is : \n");
for (i = 0; i < 5; i++) {
for (j = 0; j < 5; j++) {
    printf("%c ", playFairMatrix[i][j]);
    }
    printf("\n");
}
}

```

```

void prepareString(char *raw, char *prepared) {
int i, j = 0;
    char temp[1000];
    for (i = 0; raw[i] != '\0'; i++) {
if (isalpha(raw[i])) {        char c
= toupper(raw[i]);           if (c
== 'J') c = 'I';
temp[j++] = c;
    }
}
    temp[j] = '\0';    j = 0;    for (i
= 0; i < strlen(temp); i++) {
prepared[j++] = temp[i];    if (i
+ 1 < strlen(temp)) {        if (temp[i]
== temp[i + 1]) {
        prepared[j++] = 'X';
    }
}
}
}
}

```

```

void encrypt(char playFairMatrix[5][5]) {    char
stringToEncrypt[1000];    char
preparedString[2000];
    char encryptedString[2000];
    int i, j = 0;    printf("\nEnter the string
to Encrypt : ");
fgets(stringToEncrypt, sizeof(stringToEncrypt), stdin);
stringToEncrypt[strcspn(stringToEncrypt, "\n")] = '\0';
prepareString(stringToEncrypt, preparedString);    for (i = 0; i < strlen(preparedString);
i += 2) {    char c1 = preparedString[i];    char c2 = preparedString[i + 1];    int
r1 = row(c1,
playFairMatrix);    int c1_col = column(c1, playFairMatrix);

```

```

[j] = '\0';    if (j % 2 != 0) {    printf("Error: Encrypted text has
an odd number of letters.\n");    return;
    }    j = 0;    for (i = 0; i <
strlen(sanitizedString); i += 2) {    char c1 =
sanitizedString[i];    char c2 =
sanitizedString[i + 1];    int r1 = row(c1,
playFairMatrix);    int c1_col = column(c1,
playFairMatrix);    int r2 = row(c2,
playFairMatrix);    int c2_col =
column(c2, playFairMatrix);
if (r1 == r2) {    decryptedString[j++] = playFairMatrix[r1][(c1_col
+ 4) % 5];    decryptedString[j++] = playFairMatrix[r2][(c2_col + 4)
% 5];    } else if (c1_col == c2_col) {    decryptedString[j++] =
playFairMatrix[(r1 + 4) % 5][c1_col];    decryptedString[j++] =
playFairMatrix[(r2 + 4) % 5][c2_col];
    } else {    decryptedString[j++] =
playFairMatrix[r1][c2_col];
decryptedString[j++] = playFairMatrix[r2][c1_col];
    }
    }
    decryptedString[j] = '\0';    int hasI
= 0;    for(i = 0; decryptedString[i] !=
'\0'; i++) {    if (decryptedString[i]
== 'I') {    hasI = 1;
        break;
    }    }    if (hasI) {    printf("\nDecrypted String (with I):
%s\n", decryptedString);    char j_version[1000];    for(i =
0; decryptedString[i] != '\0'; i++) {    if (decryptedString[i]
== 'I') {    j_version[i] =
'J';
        } else {
            j_version[i] = decryptedString[i];
        }
    }
    j_version[i] = '\0';    printf("Decrypted String
(with J): %s\n", j_version);

    } else {
        printf("\nDecrypted String : %s\n", decryptedString);
    }
}

int main() {    char playFairMatrix[5][5];
createPlayFairMatrix(playFairMatrix);
displayPlayFairMatrix(playFairMatrix);    encrypt(playFairMatrix);
decrypt(playFairMatrix);
return 0;
}

#include <stdio.h>
#include <string.h>

void encryptRailFence(char *plainText, int key, char *cipherText) {
    int len = strlen(plainText);

    if (key == 1) {

```

```
strcpy(cipherText, plainText);
return;
}
```

```
char rail[key][len];
```

```
for (int i = 0; i < key; i++)
for (int j = 0; j < len; j++)
    rail[i][j] = '\n';
```

```
int row = 0, direction = 1; for
(int i = 0; i < len; i++) {
rail[row][i] = plainText[i];    row
+= direction;    if (row == key -
1 || row == 0)
    direction = -direction;
}
```

```
int k = 0; for (int i = 0; i < key;
i++) for (int j = 0; j < len; j++)
if (rail[i][j] != '\n')    cipherText[k++]
= rail[i][j];
cipherText[k] = '\0';
}
```

```
void decryptRailFence(char *cipherText, int key, char *plainText) {
    int len = strlen(cipherText);
```

```
    if (key == 1) {
strcpy(plainText, cipherText);
return;
}
```

```
char rail[key][len];
```

```
for (int i = 0; i < key; i++)
for (int j = 0; j < len; j++)
    rail[i][j] = '\n';
```

```
int row = 0, direction = 1; for
(int i = 0; i < len; i++) {
rail[row][i] = '*';    row +=
direction;    if (row == key - 1
|| row == 0)
    direction = -direction;
}
```

```
int k = 0; for (int i = 0; i
< key; i++) for (int j =
0; j < len; j++) if
(rail[i][j] == '*' && k < len)
    rail[i][j] = cipherText[k++];
```

```

        row = 0;    direction = 1;    for
(int i = 0; i < len; i++) {
    plainText[i] = rail[row][i];    row
+= direction;    if (row == key -
1 || row == 0)
        direction = -direction;
    }
    plainText[len] = '\0';
}

```

```

int main() {    printf("Program by Shubham Patil_A_29\n");
char plainText[100], cipherText[100], decryptedText[100];
int key;

```

```

    printf("Enter the plain text: ");
    fgets(plainText, 100, stdin);

```

```

    plainText[strcspn(plainText, "\n")] = '\0';

```

```

    printf("Enter the key: ");
    scanf("%d", &key);

```

```

    while(getchar() != '\n');

```

```

    encryptRailFence(plainText, key, cipherText);
    printf("Encrypted Text: %s\n", cipherText);

```

```

    decryptRailFence(cipherText, key, decryptedText);
    printf("Decrypted Text: %s\n", decryptedText);

```

```

    return 0;
}

```

Output

[Clear](#)

Program by Shubham Pati

Enter the plain text: Tulsiramji Gaikwad Patil College of Engineering and technology

Enter the key: 7

Encrypted Text: Ta EaguGilC n noyl kiofggdlsiwtlain oijaal nitnrmdPeeereha
gec

Decrypted Text: Tulsiramji Gaikwad Patil College of Engineering and technology

=== Code Execution Successful ===

Output

[Clear](#)

Enter the plain text: Department of Information Technology

Enter the key (number of columns): 5

Encrypted Text with 5 columns: Dt na nyemoftToXpefoielXan rocoXrtImnhgX

Decrypted Text with 5 columns: Department of Information Technology

Program by Krushna Rathod_A_29

=== Code Execution Successful ===

Output

[Clear](#)

Program Performance of encryption and decryption using Play Fair Cipher.

Execution By Shubham Pati A-29

Please Enter the key: MONAROHY

The Play Fair Matrix is :

M O N A R

H Y B C D

E F G I K

L P Q S T

U V W X Z

Enter the string to Encrypt : ITS INSTRUMENTS

Sanitized String: ITSINSTRUMENTS

Encrypted String : KSXSAQZDMHGMLT

Enter the string to Decrypt : ITS INSTRUMENTS

Decrypted String (with I): KSICAQKZLUGMSQ

Decrypted String (with J): KSJCAQKZLUGMSQ

=== Code Execution Successful ===

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void encryptColumnarTransposition(char *plainText, int key, char *cipherText) {
    int textLen = strlen(plainText);
    int cols = key;    int rows =
(textLen + cols - 1) / cols;

    char matrix[rows][cols];
    int k = 0;

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {            if
(k < textLen)
            matrix[i][j] =
plainText[k++];        else
matrix[i][j] = 'X';
        }
    }

    k = 0;    for (int j = 0; j < cols; j++)
{        for (int i = 0; i < rows; i++) {
cipherText[k++] = matrix[i][j];
        }
    }
    cipherText[k] = '\0';
}

k = 0;
for (int i = 0; i < rows; i++) {
for (int j = 0; j < cols; j++) {            if
(matrix[i][j] != 'X')
    plainText[k++] = matrix[i][j];
    }
}
plainText[k] = '\0';
}

int main() {
    char plainText[100], cipherText[100], decryptedText[100];
    int key;

    printf("Enter the plain text: ");    fgets(plainText,
sizeof(plainText), stdin);
    plainText[strcspn(plainText, "\n")] = '\0';

    printf("Enter the key (number of columns): ");
    scanf("%d", &key);
    encryptColumnarTransposition(cipherText, key, decryptedText);
    printf("Decrypted Text with %d columns: %s\n", key, decryptedText);

    printf("Program by Shubham Patil_A_29\n");    return
0;
}

```

```

import java.util.Scanner;

public class ExtendedEuclidean {

    public static int[] extendedEuclid(int a, int b) {        if
(b == 0) {
        return new int[] { a, 1, 0 };
    }
    int[] results = extendedEuclid(b, a % b);
    int gcd = results[0];
    int x1 = results[1];        int
y1 = results[2];

    int x = y1;        int y =
x1 - (a / b) * y1;

    return new int[] { gcd, x, y };
}

    public static void main(String[] args) {
Scanner scanner = new Scanner(System.in);
    System.out.print("Enter two numbers (a and b): ");
int a = scanner.nextInt();
    int b = scanner.nextInt();

    int[] results = extendedEuclid(a, b);
    int gcd = results[0];
int x = results[1];        int
y = results[2];

    System.out.println("Extended Euclidean Algorithm Results:");
    System.out.printf("GCD of %d and %d is: %d\n", a, b, gcd);
    System.out.printf("x = %d, y = %d\n", x, y);
    System.out.printf("Verification: %d * %d + %d * %d = %d\n", a, x, b, y, a * x + b * y);

    System.out.println("Program by Shubham Patil 29");
scanner.close();
    }
}

```

Output

Program by Krushna Rathod A_29

Enter two numbers (a and b): 5 4

--- Extended Euclidean Algorithm Results ---

GCD of 5 and 4 is: 1

Values: $x = 1$, $y = -1$

Verification: $5 * 1 + 4 * -1 = 1$

=== Code Execution Successful ===

Run

Output

Clear

Enter a prime number p: 23
Enter a primitive root g: 9
Enter private key for Alice (a): 4
Alice's public key A: 6
Enter private key for Bob (b): 3
Bob's public key B: 16
Shared secret key for Alice: 9
Shared secret key for Bob: 9
The shared secret key is successfully exchanged!

=== Code Execution Successful ===

Output

Program by Krushna Rathod A_29

Enter first prime number (p): 2

Enter second prime number (q): 3

Public Key: ($e = 3$, $n = 6$)

Private Key: ($d = 1$, $n = 6$)

Enter message to be encrypted (integer): 4023

Encrypted Message: 3

Decrypted Message: 3

=== Code Execution Successful ===

```

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>

long long gcd(long long a, long long b)
{
    while (b != 0) {
        long long temp
        = b;
        b = a % b;
        a = temp;
    }
    return a;
}

long long mod_exp(long long base, long long exp, long long mod) {
    long long result = 1;
    base = base % mod;
    while (exp > 0) {
        if
        (exp % 2 == 1) {
            result = (result * base) % mod;
        }
        exp = exp >> 1;
        base = (base * base) % mod;
    }
    return result;
}

void generate_keys(long long p, long long q, long long *e, long long *d, long long *n) {
    *n = p * q;
    long long phi =
    (p - 1) * (q - 1);

    *e = 3;
    while (gcd(*e, phi) != 1) {
        (*e)++;
    }

    long long k = 1;
    while (((1LL * k *
    phi) + 1) % *e != 0) {
        k++;
    }
    *d = ((1LL * k * phi) + 1) / *e;
}

long long encrypt(long long message, long long e, long long n) {
    return mod_exp(message, e, n);
}

long long decrypt(long long ciphertext, long long d, long long n) {
    return mod_exp(ciphertext, d, n);
}

int main() {
    printf("Program by Shubham Patil_Roll No_29\n");
    long long p, q;
    printf("
    Enter first prime number (p): ");

```

```
scanf("%lld", &p);

printf("Enter second prime number (q): ");
scanf("%lld", &q);

long long e, d, n;
generate_keys(p, q, &e, &d, &n);

printf("Public Key: (e = %lld, n = %lld)\n", e, n);
printf("Private Key: (d = %lld, n = %lld)\n", d, n);

long long message; printf("Enter message to be
encrypted (integer): ");
scanf("%lld", &message);

long long ciphertext = encrypt(message, e, n);
printf("Encrypted Message: %lld\n", ciphertext);

long long decrypted_message = decrypt(ciphertext, d, n);
printf("Decrypted Message: %lld\n", decrypted_message);

return 0;
}
```

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

long long mod_exp(long long base, long long exp, long long mod)
{
    long long result = 1;    base = base % mod;    while (exp > 0) {
    if (exp % 2 == 1) {        result = (result * base) % mod;
        }
        exp = exp >> 1;    base =
(base * base) % mod;
    }
    return result;
}

int main() {    long long p, g, a, b, A, B,
secretA, secretB;

    printf("Enter a prime number p: ");    scanf("%lld",
&p);

    printf("Enter a primitive root g: ");
scanf("%lld", &g);

    printf("Enter private key for Alice (a): ");
scanf("%lld", &a);

    A    = mod_exp(g, a, p);
printf("Alice's public key A: %lld\n", A);

    printf("Enter private key for Bob (b): ");
scanf("%lld", &b);

    B    = mod_exp(g, b, p);
printf("Bob's public key B: %lld\n", B);

    secretA = mod_exp(B, a, p);    printf("Shared secret
key for Alice: %lld\n", secretA);

    secretB = mod_exp(A, b, p);    printf("Shared
secret key for Bob: %lld\n", secretB);

    if (secretA == secretB) {        printf("The shared secret key is
successfully exchanged!\n");
    } else {
        printf("Error in key exchange.\n");
    }
    return 0;
}

```

```

#include <stdio.h>
#include <string.h>

// XOR function for binary strings void
xorStrings(char *a, char *b, char *result) {
    int len = strlen(a);    for
    (int i = 0; i < len; i++) {
        result[i] = (a[i] == b[i]) ? '0' : '1';
    }
    result[len] = '\0';
}

int main() {
    char plaintext[100], key[50], iv[50];
    char xor1[100], mac[100];

    printf("Enter Plaintext (binary): ");
    scanf("%s", plaintext);

    printf("Enter Key (binary): ");
    scanf("%s", key);

    printf("Enter Initialization Vector (IV): ");
    scanf("%s", iv);

    // Step 1: XOR plaintext with IV
    xorStrings(plaintext, iv, xor1);
    printf("\nStep 1: XOR(Plaintext, IV) = %s", xor1);

    // Step 2: XOR result with Key (encryption simulation)
    xorStrings(xor1, key, mac);
    printf("\nStep 2: XOR(Result, Key) = %s", mac);

    // Step 3: Final MAC output
    printf("\n\nFinal Message Authentication Code (MAC): %s\n", mac);

    return 0;
}

```

```

#include <stdio.h>
#include <string.h>
#include <math.h>

// Function to compute gcd long long
gcd(long long a, long long b) {
    while (b != 0) {
        long long temp = b;
        b = a % b;
        a = temp;
    }
}

```

```

    }
    return a;
}

// Modular exponentiation
long long mod_exp(long long base, long long exp, long long mod) {
    long long result = 1;    base = base % mod;    while (exp > 0) {
    if (exp % 2 == 1)
        result = (result * base) % mod;
    exp = exp >> 1;
        base = (base * base) % mod;
    }
    return result;
}

// Simple hash function (simulated) long
long simple_hash(char message[]) {
    long long hash = 0;
    for (int i = 0; i < strlen(message); i++) {
        hash = (hash + message[i]) % 1000; // simple numeric hash
    }
    return hash;
}

// RSA key generation
void generate_keys(long long p, long long q, long long *e, long long *d, long long *n) {
    *n = p * q;
    long long phi = (p - 1) * (q - 1);

    *e = 3;
    while (gcd(*e, phi) != 1) {
        (*e)++;
    }

    long long k = 1;
    while (((1LL * k * phi) + 1) % *e != 0) {
k++;
    }
    *d = (((1LL * k * phi) + 1) / *e);
}

// Sign message (using private key)
long long sign_message(long long hash, long long d, long long n) {
    return mod_exp(hash, d, n);
}

// Verify signature (using public key)
long long verify_signature(long long signature, long long e, long long n) {
    return mod_exp(signature, e, n);
}

int main() {
    printf("Program by Shubham Patil A_29\n");
}

```

```

    long long p, q, n, e, d;    printf("Enter
first prime number (p): ");    scanf("%lld",
&p);

    printf("Enter second prime number (q): ");
    scanf("%lld", &q);

    generate_keys(p, q, &e, &d, &n);

    printf("\nPublic Key: (e = %lld, n = %lld)\n", e, n);
    printf("Private Key: (d = %lld, n = %lld)\n", d, n);

    char                message[100];
    printf("\nEnter message to sign: ");
    scanf("%s", message);

    long long verified_hash = verify_signature(signature, e, n);
    printf("Verified Hash (decrypted signature): %lld\n", verified_hash);

    if (verified_hash == hash)    printf("\n✅
Signature Verified Successfully!\n");    else
        printf("\n❌ Signature Verification Failed!\n");

    return 0;
}

```

```

#include <stdio.h>
#include <string.h>

```

```

int main() {
    char plaintext[100], key[100], ciphertext[100];
    int i;

    printf("Program by Shubham Patil 29\n");
    printf("Enter the plaintext: ");
    gets(plaintext);

    printf("Enter the key (same length as plaintext): ");
    gets(key);

    // Encryption (XOR each character)
    for (i = 0; i < strlen(plaintext); i++) {
        ciphertext[i] = plaintext[i] ^ key[i]; // XOR operation
    }
    ciphertext[i] = '\0';

    printf("\nEncrypted Ciphertext: ");
    for (i = 0; i < strlen(ciphertext); i++)
        printf("%d ", ciphertext[i]); // Print ASCII values
}

```

```
// Decryption (XOR again with the same key)
for (i = 0; i < strlen(ciphertext); i++) {
    plaintext[i] = ciphertext[i] ^ key[i]; // XOR again
}
plaintext[i] = '\0';

printf("\nDecrypted Plaintext: %s\n", plaintext);

printf("\nPerfect Secrecy Achieved — Same key restores original message.\n");

return 0;
}
```

Output:-

Virtual Labs

On-line Virtual Labs

Invalid RSA public key

OK

Digitally sign the plaintext with Hashed RSA.
Plaintext (string):
 SHA-1

Hash output(hex):

Input to RSA(hex):
 Apply RSA

Digital Signature(hex):

Digital Signature(base64):

Virtual Labs

On-line Virtual Labs

Vernam Cipher and Perfect secrecy

m1:

m2:

Check Answer!!

CORRECT!!

Simulator

plaintext

key = Generate Random Key

Ciphertext

Output:-

Virtual Labs

On-line Virtual Labs

Message Authentication Code (CBC-MAC) ★★★★★

Plaintext: Next Plaintext

Key, k: Next Key

length of Initialization Vector (IV), 1, where 1 < (the length of plaintext above)/2

IV: Next IV

Put your text of size l to get the corresponding value of $F_k(\text{text})$ of size 1.

Your text: Apply Function

Function output:

Final Output: Check Answer!

Output

Program By Krushna Rathod A_29

Please enter a string: GHRCE
Enter the Key: 2

Please choose one of the following options:
1 = Encrypt the string.
2 = Decrypt the string.
2

The Decrypted string: EFPAC

=== Code Execution Successful ===

Output

Program By Krushna Rathod A_29

Please enter a string: GHRCE
Enter the Key: 5

Please choose one of the following options:
1 = Encrypt the string.
2 = Decrypt the string.
1

The Encrypted string: LMWHJ

=== Code Execution Successful ===